

Data Structures

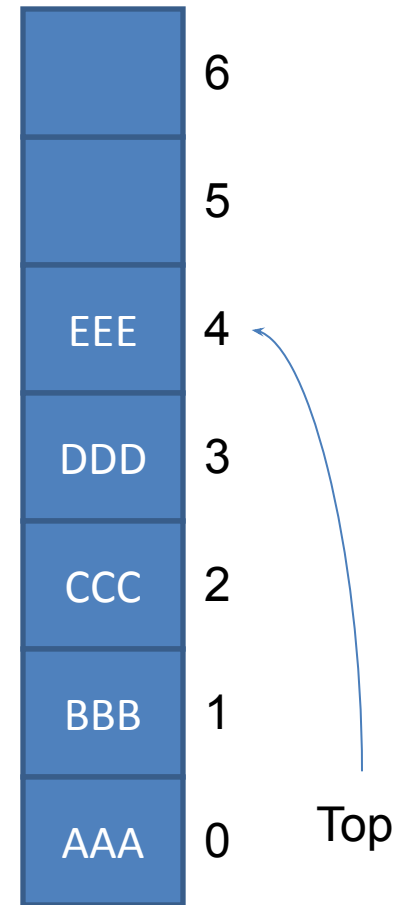
CSE207

Md. Manowarul Islam
Dept. of CSE, Jagannath University

Stack

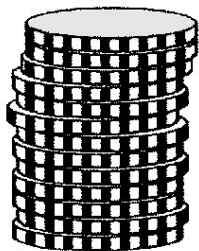
- Two basic operations are associated with stack:
- “Push” operation is used to insert an element into a stack.
- “Pop” operation is used to delete an element from a stack.

Stack: AAA, BBB, CCC, DDD, EEE

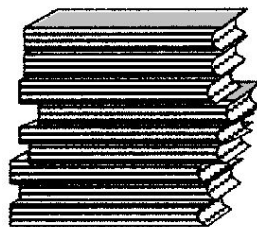


Stack

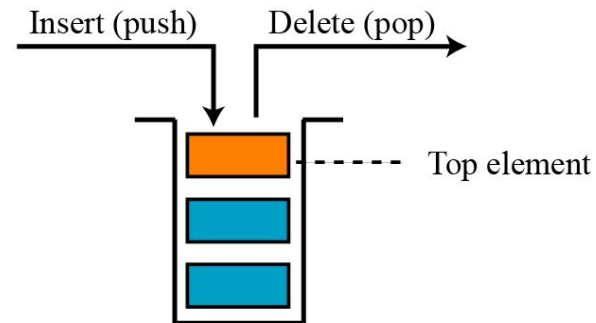
- A stack is a restricted linear list in which all additions and deletions are made at one end, the top.
- If we insert a series of data items into a stack and then remove them, the order of the data is reversed.
- This reversing attribute is why stacks are known as last in, **first out (LIFO) data structures**.



Stack of coins



Stack of books



Computer stack

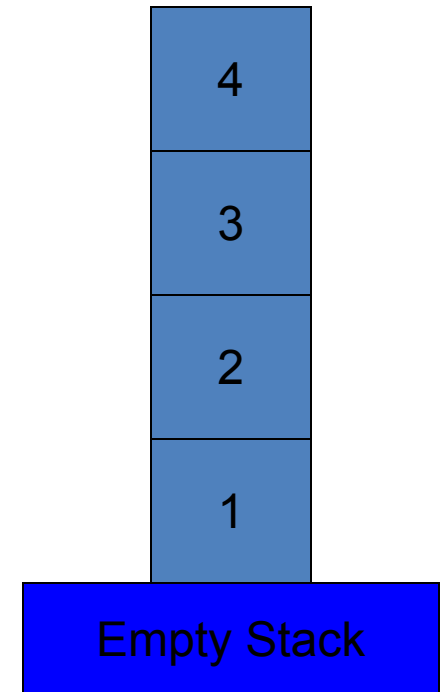
Stack

Stack has only two operation

Push: insert data on top

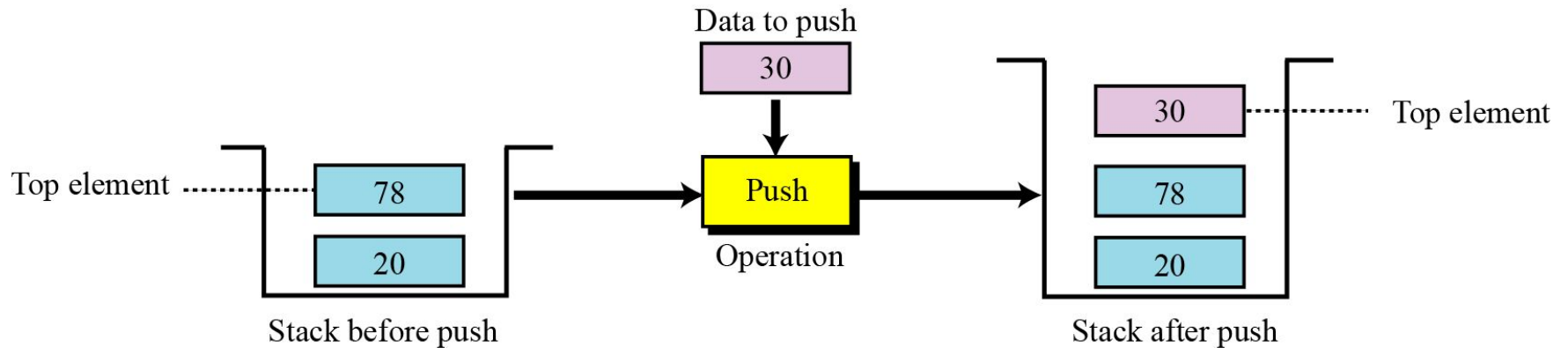
Pop: remove data from top

Also known as LIFO structure
Last In First Out



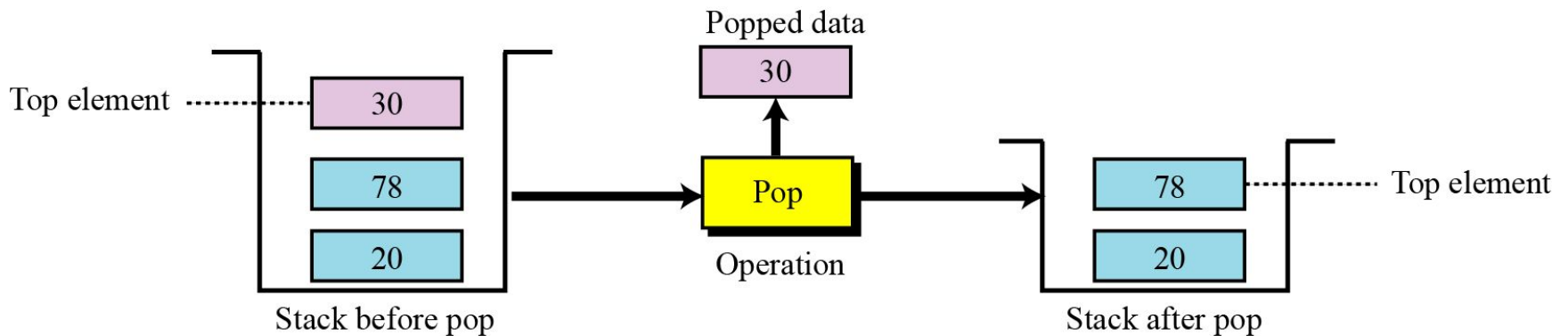
The *push* operation

- The *push* operation inserts an item at the top of the stack. The following shows the format.



The pop operation

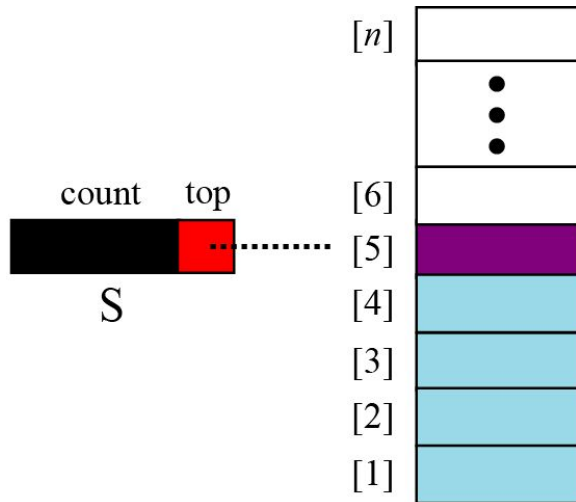
- The **pop operation** deletes the item at the top of the stack. The following shows the format.



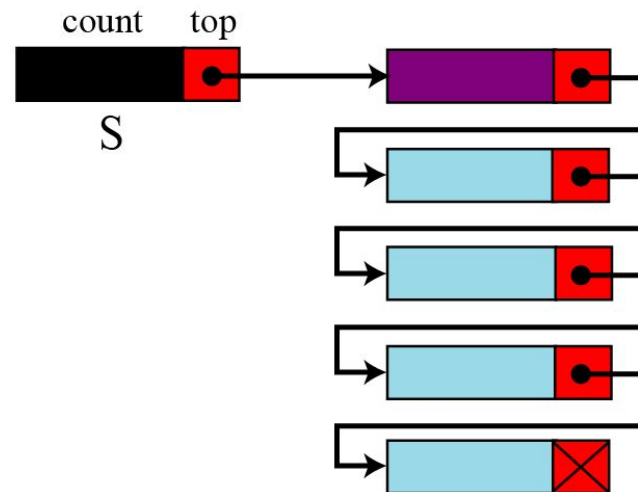
Error checking

- There are two stack errors that can occur:
 - **Underflow**: trying to pop (or peek at) an empty stack
 - **Overflow**: trying to push onto an already full stack

Implementation



b. Array implementation



c. Linked list implementation

Algorithms for Push and Pop

- **Stack is the place** where to store data.
- **Top** represents in which location the data is to be inserted.
- **MaxSTK** is the maximum size of the stack and finally, Item is the new item to be added.

```
#include <stdio.h>
int main() {
int choice = -1;

printf("MENU:\n1. Coffee\n2. Tea\n3. Exit");

while(1){
    printf("\nEnter your choice:");
    scanf("%d", &choice);

if(choice == 1){
    printf("> Enjoy your Coffee!\n");
    }
    else if (choice == 2){
    printf("> Enjoy your Tea!\n");
    }
    else if (choice == 3){
    printf("BYE!!!\n");

break;
    }
    else{
    printf("> Invalid Input\n");
    }
}
return 0;
}
```

Menu

For Push Operation

1. If Top = MaxSTK then Print: Overflow and Return. /*...Stack already filled..*/
2. Set Top := Top +1
3. Set Stack[Top] := Item
4. Return.

```
if(choice == 1){
    printf("> You choose push operation!\n");
    if(top==n-1){
        printf("\n Sorry !!! Stack Overflow..!\n");

    }
    else {

        printf("\n Enter your item: ");
        scanf("%d",&item);
        top=top+1;
        stack[top]=item;
        printf("\n %d has been Inserted !!",item);

    }

}
```

For Print Operation

1. If $\text{Top} = \text{N}-1$ then Print: Underflow and Return. /* ...Stack already filled..*/
2. For $i=0$ to Top
3. print $\text{Stack}[i]$
4. Return.

```
else if (choice == 3){
    printf("> You choose display operation!!\n");
    if(top== -1){
        printf("\n Sorry !!! Stack Underflow..\n");

    }

    else {
        for(i=0;i<=top;i++)
            printf("%d  ",stack[i]);
    }

}
```

For Pop Operation

1. If Top = -1 then Print: Underflow and Return. Set Item := Stack[Top]
2. Set Top := Top - 1
3. Return.

```
else if (choice == 2){
    printf("> You choose pop operation!!\n");
    if(top== -1){
        printf("\n Sorry !!! Stack Underflow..\n");

    }

    else{
        item=stack[top];
        printf("\n Item %d has been deleted !!! \n",item);
        top=top-1;

    }
}
```